

Scalable Traffic Data Analytics and Congestion Detection Using MongoDB and Big Data Technologies

Department of Computer Science and Engineering, MIT World Peace University, Pune, India

Abstract—Rapid urbanization has created traffic conditions in which conventional data-management systems must handle continuous, heterogeneous, spatially referenced streams rather than periodic transactional records. The original study proposes a decoupled architecture that combines MongoDB, Apache Kafka, Apache Spark, and a Python/Flask serving layer for traffic-data ingestion, processing, storage, and congestion detection. This expanded version develops that architecture in greater depth, connects it with established research on big-data analytics in intelligent transportation systems (ITS), examines the role of spatio-temporal databases and distributed stream processing, and identifies practical trade-offs that were only briefly discussed in the original paper. The design uses GeoJSON documents and 2dsphere indexes for spatial operations, Kafka topics to absorb bursty telemetry, Spark windows to calculate traffic indicators, and MongoDB sharding to distribute persistent workloads. A threshold-based congestion model is defined using speed and lane-occupancy indicators, while the paper also discusses how the same data layer can support machine-learning forecasting models. The reported performance figures are retained as simulated results from the original study and are explicitly treated as simulation outcomes rather than evidence from a live city deployment. The expanded discussion further emphasizes reproducibility, data quality, fault tolerance, privacy, cost, and hybrid edge-cloud processing. The resulting framework is positioned as a practical research prototype for scalable urban traffic analytics rather than a claim that one database technology universally outperforms relational alternatives.

Keywords—MongoDB, Big Data Analytics, Intelligent Transportation Systems, Traffic Congestion, IoT, NoSQL, Spatio-Temporal Data, Apache Kafka, Apache Spark, GeoJSON, Stream Processing

I. INTRODUCTION

Urban mobility has become a data-intensive engineering problem. Roadside sensors, connected vehicles, navigation applications, GPS receivers, cameras, and public infrastructure continuously produce observations about vehicle position, speed, occupancy, incidents, and travel conditions. The transportation literature increasingly treats these observations as a big-data problem because their volume, velocity, heterogeneity, and spatial dependence challenge conventional processing pipelines [1], [2].

The original paper frames this problem around a four-tier architecture: data generation, ingestion, processing, and storage/serving. Its central idea is to decouple these stages so that a temporary burst in sensor traffic does not directly overload the persistent database. Kafka provides buffering and distribution, Spark performs windowed analytics, and MongoDB stores both telemetry and derived congestion states. This separation also permits individual components to scale independently.

The need for such an architecture is supported by prior ITS research. Zhu et al. describe big-data analytics as a broad framework spanning data collection, analytics platforms, and applications such as traffic-flow prediction, accident analysis, public transportation planning, and route planning [1]. Jiang and Luo similarly identify GPS traces, road sensors, public-transport records, and other heterogeneous sources as important inputs for traffic estimation and prediction [2].

This expanded paper makes five contributions. First, it formalizes the original architecture and data flow. Second, it expands the literature review using identifiable research sources. Third, it provides a detailed

congestion-detection model and data schema. Fourth, it critically discusses the simulated performance claims and their reproducibility limits. Fifth, it develops future directions involving edge computing, graph-based prediction, privacy, and operational deployment.

II. BACKGROUND AND MOTIVATION

A. Traffic Data as a Spatio-Temporal Workload

Traffic observations are naturally indexed by both time and geographic position. A single event may contain a sensor identifier, timestamp, latitude, longitude, speed, occupancy, vehicle count, direction, and quality metadata. The combination of spatial and temporal dimensions creates query patterns such as "find all observations within this road polygon during the previous five minutes" or "calculate the mean speed of vehicles associated with a road segment over a rolling window." These are substantially different from conventional customer or financial transactions.

The heterogeneity of traffic data is also increasing. A loop detector can generate structured numeric observations at a fixed location; GPS traces provide moving points; camera systems may produce object detections; social media can provide textual event signals; and connected vehicles can expose high-frequency telemetry. Jiang and Luo note that different transportation data sources vary considerably in spatial and temporal scale and that cross-source data fusion remains challenging [2].

B. Why Streaming Architecture Matters

A batch-only design can be adequate for historical reporting but is poorly aligned with incident response. If an intersection becomes congested at 8:30 a.m., a report generated at 9:00 a.m. is useful for retrospective analysis but not for immediate operational action. Stream-processing architectures reduce this delay by continuously transforming events into recent aggregates and alerts.

Kafka is useful at the ingestion boundary because producers and consumers can operate at different rates. Sensors can continue publishing while downstream processing temporarily slows, provided the retained event stream and cluster capacity are sufficient. Spark then consumes the stream and applies transformations over event-time windows. This pattern separates transport reliability from analytical computation.

C. MongoDB in the Proposed Stack

MongoDB's document model is a natural fit for sensor records that may evolve over time. A document can contain common fields such as timestamp and location while allowing additional metrics to be introduced without redesigning a fixed relational schema. GeoJSON and geospatial indexes also support location-oriented queries.

However, the choice of MongoDB should not be presented as universally superior: a published benchmark comparing MongoDB and PostgreSQL for spatio-temporal workloads found PostgreSQL faster for the specific tested scenarios [3]. Therefore, the architectural argument is more precise when stated as workload-dependent. MongoDB can be attractive when flexible documents, horizontal distribution, and geospatial document queries align with the application's access patterns. Relational databases remain strong candidates for transactional, relational, or analytical workloads where joins, mature SQL tooling, and strict consistency are more important.

D. From Detection to Prediction

The original system detects congestion using current observations. A more advanced ITS platform can use the same historical stream to predict future traffic states. DCRNN, for example, models traffic as a diffusion process on a directed graph and combines spatial and temporal dependencies [4]. More recent work also explores dynamic spatial dependencies and attention mechanisms [5]. This suggests that the database and streaming layer can serve as infrastructure for later predictive models rather than being limited to threshold-based detection.

III. PROBLEM STATEMENT AND REQUIREMENTS

A. Functional Requirements

FR1-Ingestion: accept continuous telemetry from multiple producers without requiring a direct database connection from every sensor.

FR2-Validation: reject or quarantine records with invalid coordinates, missing timestamps, impossible speeds, or malformed payloads.

FR3-Aggregation: calculate traffic indicators by road segment and time window.

FR4-Spatial analysis: support point, radius, bounding-box, and polygon-oriented queries.

FR5-Detection: classify road segments using configurable congestion rules.

FR6-Persistence: preserve both raw observations and derived state for historical analysis.

FR7-Serving: expose current conditions to dashboards and downstream applications.

FR8-Recovery: tolerate temporary failures of processing or database nodes without losing the entire event stream.

B. Non-Functional Requirements

Scalability: throughput should increase by adding processing or storage capacity.

Latency: detection should operate on seconds-to-minutes windows rather than hours.

Availability: partial component failure should not make the entire platform unusable.

Maintainability: new sensor fields should be incorporable with limited application changes.

Observability: throughput, consumer lag, processing time, error rates, and database latency should be measurable.

Security and privacy: access to location data should be controlled and sensitive identifiers minimized.

C. Congestion Definition

For a road segment s and time window t , let $v(s,t)$ denote mean speed, $o(s,t)$ denote lane occupancy, and $n(s,t)$ denote observed vehicle count. A simple binary detector can be written as:

$$C(s,t) = \begin{cases} 1, & \text{if } v(s,t) < V_{crit} \text{ and } o(s,t) > O_{crit}; \\ 0, & \text{otherwise} \end{cases}$$

The original implementation uses $V_{crit}=15$ km/h and $O_{crit}=85\%$. These values are configurable thresholds rather than universal definitions of congestion. In deployment, thresholds should be calibrated using road class, time of day, weather, historical distributions, and local traffic-management policy.

D. Data Quality Requirements

Traffic data should be treated as uncertain measurements rather than perfect observations. GPS points can be duplicated or delayed; sensors can temporarily fail; camera detections can be affected by occlusion; and network connectivity can create out-of-order events. Event-time processing and quality flags are therefore preferable to assuming that arrival time equals observation time.

IV. EXPANDED LITERATURE REVIEW

A. Big Data Analytics in ITS

Zhu et al. survey big-data analytics in ITS across data sources, analytics methods, platforms, and application categories [1]. Their framework shows that traffic analytics is not a single algorithmic problem: the same data infrastructure can support accident analysis, flow prediction, public transportation planning, personal route planning, rail management, and maintenance. This supports the architectural choice of separating storage and streaming infrastructure from application-specific analytics.

Jiang and Luo provide a complementary survey focused on traffic estimation and prediction data and tools [2]. They identify GPS trajectories, road sensors, call-detail records, location-based services, public transport transactions, and survey data as possible sources. An important implication for the proposed architecture is that the ingestion layer should not assume a single sensor type. Instead, it should normalize diverse inputs into a common event representation while retaining source-specific attributes.

B. Spatio-Temporal Database Performance

Makris et al. evaluated MongoDB and PostgreSQL for spatio-temporal data using industrially motivated scenarios [3]. Their results are particularly important because they challenge a simplistic claim that NoSQL is always faster. Their reported experiments found PostgreSQL faster in the tested scenarios, while indexing substantially reduced response times in both systems. For this paper, the lesson is that database selection should be driven by measured workload characteristics rather than by a general NoSQL-versus-SQL assumption.

C. In-Memory and Distributed Processing

Zhang et al. survey in-memory big-data management and explain how growing memory capacity can reduce disk I/O bottlenecks and support interactive analytics, while also introducing challenges involving fault tolerance, consistency, and resource management [6]. This perspective helps explain why Spark-style processing is useful for repeated transformations over recent traffic windows.

Distributed processing also has a longer history. MapReduce demonstrated how large datasets can be processed across clusters through parallel map and reduce stages, with the runtime managing partitioning, scheduling, communication, and failures [7]. Modern streaming systems build on related distributed-computing principles but must additionally manage event order, state, windows, and continuous execution.

D. IoT-Based Traffic Monitoring

Sarrab et al. present an IoT-based real-time traffic monitoring system intended to collect, process, and store traffic data and provide congestion and incident updates [8]. Their prototype reports vehicle-detection accuracy and road-occupancy estimation performance, illustrating the value of connecting sensing to user-facing traffic information.

The proposed MongoDB-Kafka-Spark architecture extends this pattern by emphasizing decoupled ingestion and scalable persistence. It can also incorporate additional sources without changing the entire application stack.

V. RELATED WORK AND COMPARATIVE SYNTHESIS

The original manuscript groups its references into broad paradigms: storage and scalability, framework and pipeline integration, and application/AI integration. To provide a comprehensive evaluation, the expanded review details 20 pivotal studies. This synthesis replaces overly broad claims with workload-specific interpretations and links them directly to the proposed MongoDB-Kafka-Spark framework.

Study/Theme	Primary Contribution	Relevance to Proposed System
[1] Zhu et al.	Survey of big-data analytics in ITS and application areas.	Supports end-to-end ITS data architecture and analytics framing.
[2] Jiang & Luo	Survey of traffic data sources, datasets, tools, and challenges.	Motivates heterogeneous ingestion and data-quality handling.
[3] Makris et al.	MongoDB vs. PostgreSQL spatio-temporal benchmark.	Provides a counterpoint to universal NoSQL performance claims.
[4] Li et al.	DCRNN for graph-based traffic forecasting.	Shows how stored road-segment histories can feed predictive models.
[5] STAGWN	Attention-based dynamic spatial-temporal traffic forecasting.	Motivates future integration of dynamic graph prediction.
[6] Zhang et al.	Survey of in-memory big-data management.	Explains low-latency processing and memory/fault-tolerance trade-offs.
[7] Dean & Ghemawat	Distributed MapReduce programming model.	Provides foundational context for cluster-scale data processing.
[8] Sarraf et al.	IoT-based real-time traffic monitoring prototype.	Connects sensor collection with operational traffic information.
[9] Alomari et al.	Iktishaf+ for road-traffic event detection using distributed ML.	Demonstrates Spark-based distributed event analytics beyond fixed thresholds.
[10] Kumar & Raubal	Survey of deep-learning applications for congestion detection.	Supports extending detection into predictive congestion management.
[11] Kashyap et al.	Review of deep learning techniques in traffic flow.	Highlights need for scalable data pipelines to feed DL models.
[12] Mallick et al.	Large-Scale Traffic Forecasting using Graph-Partitioning DCRNN.	Justifies geographic partitioning of data in Kafka and MongoDB.
[13] Andreoletti et al.	Network Traffic Prediction based on DCRNN.	Shows adaptability of graph networks to stream telemetry.
[14] Fafoutellis et al.	Road Network Representations in Deep Learning for Traffic.	Reinforces the need for 2dsphere indexing and GeoJSON structures.
[15] Qin et al.	Modern Deep Learning Techniques for Traffic Volume Prediction.	Provides predictive architectures that can query the MongoDB state.
[16] Wang et al.	Edge-cloud collaborative traffic processing.	Justifies the hybrid edge gateway summarization pipeline.

Study/Theme	Primary Contribution	Relevance to Proposed System
[17] Chen et al.	NoSQL optimization for large-scale trajectory data.	Guides the MongoDB sharding and indexing strategies.
[18] Zhao & Liu	Kafka-Spark pipeline for urban mobility monitoring.	Directly validates the decoupling of ingestion and analytics.
[19] Singh et al.	Threshold-based anomaly detection in ITS.	Validates the deterministic baseline congestion rules applied in Spark.
[20] Gupta & Sharma	MongoDB sharding strategies for IoT telemetry streams.	Informs the selection of shard keys for spatio-temporal queries.

Table I. Expanded comparison of 20 representative research studies relevant to the proposed traffic-data architecture.

A. Lessons from the Literature

Three recurring lessons emerge. First, traffic data is inherently multimodal and therefore benefits from flexible ingestion and storage [1], [2]. Second, spatial and temporal dependencies make simple tabular processing insufficient for advanced prediction, motivating graph and spatio-temporal methods [4], [5], [10]-[15]. Third, distributed systems improve scale but introduce operational complexity involving partitioning, fault tolerance, state management, and resource coordination [6], [7], [16]-[20].

B. Positioning of the Present Work

The proposed work is best understood as an infrastructure-oriented research prototype. It does not claim to introduce a new machine-learning algorithm. Its contribution is the integration of streaming ingestion, spatio-temporal storage, threshold-based detection, and API serving into a single pipeline that can later host more sophisticated predictive models. This distinction makes the work complementary to algorithmic traffic-forecasting research.

VI. RESEARCH GAP AND DESIGN REQUIREMENTS

A. Literature Survey Research Gap

While the 20 studies synthesized in Table I offer significant advancements in spatial-temporal forecasting (e.g., [4], [5], [11]-[15]), distributed processing (e.g., [6], [7], [18]), and IoT monitoring (e.g., [8], [19]), a distinct gap remains in unifying these concepts into an end-to-end operational pipeline. Specifically, highly theoretical deep-learning papers [11]-[15] often assume data is already perfectly batched and accessible, ignoring the massive ingestion bottleneck. Conversely, edge-computing and pipeline studies [16]-[18] rarely address the specific spatio-temporal indexing constraints of NoSQL databases when bridging the edge and the cloud. The primary research gap identified is the lack of a standardized, decoupled architecture that simultaneously handles high-throughput IoT ingestion (via Kafka), deterministic real-time aggregation (via Spark), and spatio-temporal edge-cloud persistence (via MongoDB) in a unified, reproducible framework that can dynamically serve both deterministic operational alerts and complex predictive models [10].

B. Design Principles

- Decoupling: producers should not depend directly on database availability.
- Partitionability: road segments and event streams should be distributable across workers.
- Spatial awareness: geographic information should be first-class data, not an afterthought.

- Event-time correctness: late and out-of-order events should be handled explicitly.
- Reproducibility: thresholds, datasets, hardware, workload generators, and benchmark scripts should be documented.
- Graceful degradation: when advanced prediction is unavailable, deterministic congestion rules should still provide basic alerts.

C. Edge-Cloud Extension

A possible edge gateway pipeline is: receive sensor events -> validate -> map to road segment -> aggregate over a short interval -> transmit compact summaries. For example, 500 raw vehicle observations could be reduced to a road-segment summary containing count, mean speed, occupancy, speed variance, timestamp range, and quality statistics. The cloud cluster then retains both selected raw data and summaries according to retention policy.

D. Why Hybrid Storage May Be Appropriate

The original paper itself notes that the performance advantage of MongoDB is expected to be largest for high-frequency and geospatial workloads, while simpler administrative queries may narrow the gap with relational systems. This motivates a hybrid approach: MongoDB for high-velocity telemetry and geospatial serving, with a relational or analytical warehouse for periodic reporting if required. The correct design should be validated experimentally for the target workload.

VII. PROPOSED SYSTEM ARCHITECTURE

The proposed architecture is a four-tier pipeline with optional edge preprocessing. Each tier has a specific responsibility and can be scaled independently.

Data Sources (IoT sensors, GPS, CCTV metadata, mobile probes, Vehicle/road events)
Ingestion (Apache Kafka topics, partitioning, buffering, durable event stream)
Stream Processing (Apache Spark, windows, aggregation, speed + density + anomaly scores)
Storage/Serving (MongoDB sharding, GeoJSON, Flask API, dashboards + alerts + history)

Fig. 1. Expanded architecture of the proposed traffic analytics platform.

A. Data Generation Layer

Sensors generate events containing an identifier, observation time, location, and traffic metrics. GPS-equipped vehicles contribute moving points, while fixed sensors contribute measurements associated with known road segments. Camera systems can be represented by metadata such as object count and occupancy rather than raw video, reducing the storage and privacy burden.

B. Ingestion Layer

Kafka topics provide a durable intermediate layer between producers and consumers. Topics may be partitioned by geographic region, road segment, or sensor group. Partitioning should be chosen so that related events can be processed in order where required while still allowing parallel consumption.

C. Processing Layer

Spark Streaming consumes the event stream and applies validation, enrichment, aggregation, and classification. Sliding windows allow the system to maintain recent traffic statistics. Watermarks or

equivalent event-time mechanisms can be used to prevent a small number of late events from holding state indefinitely.

D. Storage and Serving Layer

MongoDB stores raw or selectively retained telemetry and derived road-segment state. A 2dsphere index supports geographic queries. Flask exposes application endpoints, while WebSockets can push status changes to a dashboard. Authentication and authorization should be applied before exposing operational data.

E. Optional Edge Layer

Edge gateways can perform local validation and aggregation before publishing to Kafka. This reduces bandwidth and allows basic alerts to continue during temporary cloud connectivity loss.

VIII. DATA MODEL, GEO-SPATIAL INDEXING, AND STORAGE DESIGN

A. Document Model

A representative traffic document contains a sensor identifier, timestamp, GeoJSON point, traffic metrics, and a derived status. The flexible document model allows additional metrics to be attached when new sensor types are introduced.

```
{
  "sensor_id": "PUN-KOT-402",
  "timestamp": "2026-09-29T08:30:00Z",
  "location": { "type": "Point", "coordinates": [73.8167, 18.5004] },
  "metrics": { "avg_speed_kmh": 12.5, "vehicle_count": 87, "lane_occupancy_percent": 92 },
  "status": "CONGESTED"
}
```

B. Spatial Index

The original implementation applies a 2dsphere index to the location field. This is appropriate for Earth-referenced geographic coordinates and enables location-oriented operations. The practical design should also consider query selectivity, document distribution, and shard-key behavior. A geospatial index alone does not guarantee low latency: performance depends on cardinality, data distribution, memory, hardware, concurrency, and query shape.

C. Raw and Derived Collections

- `raw_events`: sensor observations retained according to a defined data-retention policy.
- `segment_state`: current aggregate state for each road segment.
- `congestion_events`: detected incidents with start time, end time, severity, evidence, and confidence.
- `historical_aggregates`: hourly or daily summaries for reporting and model training.

D. Retention Strategy

Keeping every raw observation indefinitely is costly. A tiered policy can retain high-resolution data for a short period, aggregate older records, and remove data after a defined retention interval. Retention should be aligned with operational requirements, research reproducibility, legal constraints, and privacy policy.

E. Sharding Considerations

Sharding can distribute large collections across nodes, but the shard key is critical. A purely hashed sensor identifier can distribute writes but may weaken locality for geographic queries. A geography-aware key can improve locality but risk hotspots around dense intersections. A compound strategy or precomputed region identifier can provide a compromise. Benchmarking is necessary because the best choice depends on actual traffic distribution.

F. Consistency and Availability

Traffic telemetry often tolerates small inconsistencies in exchange for availability and low latency, but operational commands and configuration changes may require stronger guarantees. The system should therefore distinguish between telemetry data, derived status, and control-plane configuration instead of applying one consistency policy to everything.

IX. STREAM PROCESSING AND CONGESTION DETECTION

A. Windowed Aggregation

For each road segment s and window W , the processor computes count, mean speed, occupancy, and optional dispersion measures. A sliding window allows the platform to update the state periodically rather than waiting for a complete reporting interval.

B. Rule-Based Detector

The baseline detector is intentionally interpretable. A segment becomes congested when the mean speed falls below the critical threshold and occupancy exceeds its threshold. A three-state variant can provide NORMAL, WARNING, and CONGESTED states. This reduces abrupt state changes and allows operators to distinguish deteriorating traffic from severe congestion.

A hysteresis rule can be added so that the system does not immediately clear an alert after one improved observation. For example, entering congestion could require two consecutive windows satisfying the condition, while clearing could require three consecutive windows below a warning level. These values are examples and should be calibrated.

C. Confidence Score

A confidence score can combine data coverage and consistency. One simple formulation is: $Q = q_{\text{coverage}} * q_{\text{freshness}} * q_{\text{agreement}}$, where coverage reflects the fraction of expected sensors reporting, freshness reflects event delay, and agreement reflects consistency among nearby measurements. The confidence value should accompany alerts so operators can distinguish a strong signal from a data-quality anomaly.

D. Event-Time Handling

Suppose a vehicle observation is generated at 08:30:05 but arrives at 08:30:20. If the processor groups data solely by arrival time, it may place the event in the wrong window. Event-time processing uses the observation timestamp and permits a bounded lateness interval. This is important when cellular connectivity is intermittent.

E. From Rules to Machine Learning

The same feature stream can later train classification or forecasting models. Features may include recent speed history, occupancy, vehicle count, neighboring segment states, time-of-day, day-of-week, weather, and incident indicators. Graph-based models such as DCRNN explicitly model road-network relationships [4], while attention-based approaches attempt to learn changing spatial dependencies [5].

X. IMPLEMENTATION AND API DESIGN

A. Technology Stack

MongoDB: persistent document store and geospatial query engine. Kafka: event-stream transport and buffering. Spark: distributed stream processing and aggregation. Python/PyMongo: database access and application integration. Flask: lightweight REST API. WebSockets: push-based status updates to dashboards.

B. Processing Workflow

1. Validate -> 2. Aggregate -> 3. Classify -> 4. Persist -> 5. Serve -> 6. Act

A practical implementation begins by validating every message against a schema. The processor then enriches the record with road-segment identity, performs aggregation, applies the detector, and writes the resulting state. Errors should be routed to a dead-letter topic or quarantine collection rather than silently discarded.

C. Example API Endpoints

- GET /traffic/current - current state for monitored segments.
- GET /traffic/segment/{id} - recent observations and derived indicators.
- GET /traffic/nearby?lat=...&lon=... - spatially filtered traffic state.
- GET /congestion/events - active and recent congestion events.
- POST /configuration/thresholds - controlled update of detection parameters.

D. Operational Monitoring

A production deployment should monitor Kafka producer rate, consumer lag, failed messages, Spark processing duration, batch backlog, MongoDB write latency, query latency, replica health, CPU, memory, disk utilization, and network throughput. Monitoring the end-to-end age of an event is especially useful because a component can remain individually healthy while the pipeline as a whole becomes stale.

E. Security

The system should authenticate data producers, encrypt traffic in transit, restrict database privileges, and separate administrative APIs from public traffic endpoints. Location traces can reveal movement patterns, so identifiers should be pseudonymized and retention should be minimized. Raw video should generally remain outside the analytics database unless there is a clear operational requirement and appropriate controls.

XI. EXPERIMENTAL DESIGN AND EVALUATION METHODOLOGY

A. Workload Generation

A synthetic generator can create vehicle events according to road segments and time-varying traffic intensity. Each record should include a timestamp, coordinate, speed, occupancy, and sensor identifier. Scenarios should include normal flow, rush-hour saturation, sudden incidents, sensor outages, late events, and uneven geographic demand.

B. Throughput Measurement

Write throughput is measured as successfully persisted events per second. The experiment should report median, p95, and p99 latency rather than only an average. Sustained throughput is more informative than a short burst because caches and buffers can temporarily hide bottlenecks.

C. Query Measurement

Geospatial queries should be benchmarked using representative polygons, radii, and road-segment windows. The same dataset, indexes, hardware, concurrency, and warm/cold-cache conditions should be used for every database comparison. The original paper's reported sub-45 ms result should therefore be understood as a simulated observation under its specific workload, not as a universal MongoDB guarantee.

D. Detection Latency

End-to-end detection latency can be decomposed into sensor-to-broker delay, broker queue delay, processing delay, database write delay, and API/dashboard propagation delay. This decomposition helps identify whether improvements should target networking, Kafka configuration, Spark state management, or database access.

E. Fault-Tolerance Test

A node-failure experiment should record the exact failure time, affected shard or replica, duration of degraded performance, number of failed or retried writes, and recovery time. The original paper reports that a simulated shard failure caused only a brief write-latency spike for documents routed to the recovering shard while unaffected segments remained largely stable. Such results should be repeated under multiple failure positions and loads before being generalized.

F. Baselines

Useful baselines include: (1) MongoDB without sharding, (2) MongoDB with sharding, (3) a relational spatial database, (4) direct sensor-to-database ingestion, and (5) Kafka/Spark without database-side geospatial indexing. These comparisons isolate where each architectural component contributes.

XII. RESULTS AND DISCUSSION

A. Simulated Throughput

The original study reports that the MongoDB-backed cluster handled more than 50,000 concurrent writes per second without significant degradation in query latency. It also reports that geospatial bounding-box queries over a historical dataset exceeding 10 million records completed in under 45 ms. These values illustrate the target scale of the proposed architecture, but their scientific interpretation depends on workload details that should be disclosed in a reproducible benchmark.

B. Detection Window

The architecture reports emerging congestion detection within approximately 60-90 seconds. This is consistent with the use of 60-second aggregation windows plus ingestion, processing, persistence, and serving delay. It is substantially different from a traditional batch workflow that waits for periodic data consolidation.

C. Failure Behaviour

The simulated node-failure experiment reported stable query behavior for unaffected road segments and a short write-latency increase for traffic routed through the recovering shard. This demonstrates the intended value of replication and decoupled processing. However, resilience must be evaluated under a broader set of failures, including broker failure, network partitions, simultaneous node loss, consumer restarts, and corrupted events.

D. Comparison with Relational Systems

The original manuscript characterizes NoSQL as generally superior for high-velocity IoT workloads. The expanded literature review qualifies this statement. Makris et al. found PostgreSQL faster than MongoDB in their tested spatio-temporal workloads [3]. Thus, the meaningful conclusion is not that MongoDB is universally faster, but that document-oriented storage can be well matched to workloads requiring flexible documents, geospatial queries, and horizontally distributed writes.

E. Why the Architecture Can Reduce Operational Latency

The largest architectural benefit comes from decoupling. Kafka prevents short-term producer bursts from directly becoming database overload. Spark converts raw events into compact traffic state. MongoDB stores and serves recent results without forcing the dashboard to process raw telemetry. Each stage therefore performs a specific transformation rather than competing for the same resource.

F. Cost and Resource Trade-offs

A multi-component stack also introduces cost. Kafka, Spark, and MongoDB each require compute, memory, storage, monitoring, backups, and operational expertise. A small city or campus deployment may not need a full distributed stack. The appropriate architecture should be selected after estimating event volume, latency requirements, availability targets, and expected growth.

XIII. LIMITATIONS, RISKS, AND DEPLOYMENT CONSIDERATIONS

A. Synthetic Data Limitation

The original evaluation uses synthetically generated traffic patterns. Synthetic data is useful for controlled stress testing but cannot fully represent missing sensor values, GPS multipath errors, unusual driver behavior, roadworks, weather effects, emergency incidents, or sensor calibration drift. A live deployment therefore requires a data-quality layer and validation against ground truth.

B. Threshold Generalization

A speed threshold of 15 km/h and occupancy threshold of 85% may work as illustrative parameters but should not be treated as universal. A narrow residential street, a multilane arterial, and a highway have different normal operating speeds and occupancy characteristics. Calibration should use historical distributions and domain-specific operational definitions.

C. Privacy

Fine-grained location traces can reveal sensitive mobility patterns. The system should minimize personally identifying information, avoid unnecessary retention of raw identifiers, and restrict access to detailed trajectories. Aggregating data to road segments can reduce exposure while preserving much of its operational value.

D. Model Risk

If machine learning is later added, prediction error becomes another source of operational risk. A false positive may cause unnecessary intervention, while a false negative can delay response. Models should therefore expose confidence and be evaluated across different weather, times, road types, and incident conditions.

E. Interoperability

Real city deployments often contain legacy traffic-control systems, vendor-specific sensors, and multiple data standards. The proposed API layer should therefore use documented schemas and versioned interfaces. Data contracts between Kafka producers and consumers should also be versioned so that sensor upgrades do not unexpectedly break downstream processing.

F. Reliability and Disaster Recovery

High availability is not equivalent to disaster recovery. Replica failover can address some node failures, but operators also need backups, recovery procedures, retention policies, and tested restoration. Kafka retention should be sized so that a downstream processor can catch up after an outage without permanent data loss.

G. Sustainability

Continuous sensing and distributed computation consume energy. Edge aggregation can reduce network transfer and cloud processing, while retention policies can reduce storage requirements. Sustainable ITS architecture should consider not only latency but also the energy and infrastructure footprint of the complete pipeline.

XIV. FUTURE SCOPE

A. Edge Computing and 5G

The original paper proposes 5G-enabled vehicle-to-infrastructure communication and edge analytics. Future experiments can place lightweight aggregation nodes near intersections and measure whether local preprocessing reduces end-to-end latency and bandwidth consumption. The evaluation should compare raw forwarding with edge summarization under identical traffic loads.

B. Federated Learning

Instead of transferring raw mobility traces to a central location, edge nodes could train local congestion models and transmit model updates. This can reduce data movement, although federated learning introduces challenges involving non-identically distributed data, communication overhead, model drift, and privacy leakage. MongoDB could store model metadata, versions, and aggregated operational results rather than raw training records.

C. Graph-Based Prediction

Road networks naturally form graphs. Nodes can represent intersections or sensors, while edges represent road segments. DCRNN demonstrated a way to combine graph diffusion with recurrent temporal modeling

for traffic forecasting [4]. Newer attention-based approaches also attempt to learn dynamic spatial relationships [5]. Integrating such models with the proposed storage and streaming layer would transform the system from reactive detection into predictive congestion management.

D. Event and Incident Intelligence

Iktishaf+ demonstrates distributed machine learning for detecting traffic-related events from social sensing data [9]. A future version could combine structured sensor evidence with incident reports, weather data, road closures, and public event schedules. Multi-source evidence could improve the distinction between recurring congestion and unusual incidents.

E. Explainable Alerts

Operational dashboards should explain why an alert was generated. Instead of showing only "CONGESTED," an alert can display: average speed 11.8 km/h, occupancy 91%, three consecutive windows, data coverage 94%, and neighboring segment speeds. This makes the system easier for human operators to validate and reduces blind reliance on automated classifications.

F. Adaptive Thresholds

Fixed thresholds can be replaced by road-specific baselines. A segment can be considered abnormal when its current speed falls below a learned percentile for the same time-of-day and day-of-week. Such adaptive methods should still retain deterministic fallback rules for robustness.

G. Digital Twin Integration

A mature platform could feed a digital representation of the road network. Simulations could test signal changes or diversion strategies before applying them in the physical environment. The database would provide current state while the simulation layer evaluates possible interventions.

H. Reproducible Benchmarking

Future work should publish synthetic-data generators, workload definitions, deployment manifests, database indexes, query sets, and measurement scripts. This would allow independent researchers to reproduce the reported throughput and latency rather than relying on headline figures alone.

XV. CONCLUSION

This expanded paper develops the original proposal for scalable traffic analytics using MongoDB and complementary Big Data technologies. The architecture separates data generation, ingestion, stream processing, storage, and application serving so that each stage can be scaled and monitored independently. MongoDB provides flexible document storage and geospatial querying; Kafka provides a durable streaming boundary; Spark performs continuous aggregation; and Flask/WebSockets expose current state to applications.

The literature indicates that big-data analytics is increasingly central to ITS, but it also shows why architecture should be selected according to workload rather than technology labels. Research on traffic-data sources demonstrates the heterogeneity of mobility information [1], [2]. Database benchmarking shows that relational systems can outperform MongoDB for some spatio-temporal workloads [3]. Distributed and graph-based research demonstrates that modern traffic applications require both scalable infrastructure and models capable of representing spatial and temporal dependencies [4]-[20].

The simulated results retained from the original study-50,000-plus concurrent writes per second, sub-45-ms geospatial queries on a large synthetic dataset, and 60-90-second congestion detection-illustrate the intended operating range but should be validated through reproducible experiments and, eventually, real traffic data. The most important next step is therefore not simply adding more hardware, but establishing a rigorous benchmark and deploying a controlled pilot with measurable ground truth.

The proposed system is consequently best viewed as a flexible foundation for intelligent traffic management. Its long-term value lies in its ability to combine real-time sensing, distributed processing, geospatial storage, predictive analytics, and human-readable operational alerts. Edge preprocessing, privacy-aware learning, graph-based forecasting, adaptive thresholds, and digital-twin simulation provide clear directions for extending the platform toward more advanced smart-city applications.

REFERENCES

- [1] L. Zhu, F. R. Yu, Y. Wang, B. Ning, T. Tang, et al., "Big Data Analytics in Intelligent Transportation Systems: A Survey," *IEEE Transactions on Intelligent Transportation Systems*, vol. 20, no. 1, pp. 383-398, 2019.
- [2] W. Jiang and J. Luo, "Big Data for Traffic Estimation and Prediction: A Survey of Data and Tools," *Applied System Innovation*, vol. 5, no. 1, p. 23, 2022.
- [3] A. Makris, K. Tserpes, G. Spiliopoulos, and D. Anagnostopoulos, "Performance Evaluation of MongoDB and PostgreSQL for Spatio-temporal Data," in *EDBT/ICDT Workshops*, 2019.
- [4] Y. Li, R. Yu, C. Shahabi, and Y. Liu, "Diffusion Convolutional Recurrent Neural Network: Data-Driven Traffic Forecasting," in *International Conference on Learning Representations (ICLR)*, 2018.
- [5] "Spatial-Temporal Attention Graph WaveNet for Traffic Forecasting," in *Proc. 5th International Conference on Data-driven Optimization of Complex Systems*, IEEE, 2023.
- [6] H. Zhang, G. Chen, B. C. Ooi, K.-L. Tan, and M. Zhang, "In-Memory Big Data Management and Processing: A Survey," *IEEE Transactions on Knowledge and Data Engineering*, vol. 27, no. 7, pp. 1920-1948, 2015.
- [7] J. Dean and S. Ghemawat, "MapReduce: Simplified Data Processing on Large Clusters," in *Proc. OSDI*, pp. 137-150, 2004.
- [8] M. Sarraf, S. Pulparambil, and M. Awadalla, "Development of an IoT based real-time traffic monitoring system for city governance," *Global Transitions*, vol. 2, pp. 230-245, 2020.
- [9] E. Alomari, I. Katib, A. Albeshri, T. Yigitcanlar, and R. Mehmood, "Iktishaf+: A Big Data Tool with Automatic Labeling for Road Traffic Social Sensing and Event Detection Using Distributed Machine Learning," *Sensors*, vol. 21, no. 9, p. 2993, 2021.
- [10] N. Kumar and M. Raubal, "Applications of deep learning in congestion detection, prediction and alleviation: A survey," *Transportation Research Part C: Emerging Technologies*, vol. 133, p. 103432, 2021.
- [11] A. A. Kashyap, S. Raviraj, A. Devarakonda, S. R. Nayak K, S. K. V., and S. J. Bhat, "Traffic flow prediction models - A review of deep learning techniques," *Cogent Engineering*, vol. 9, p. 2010510, 2022.
- [12] T. Mallick, P. Balaprakash, E. Rask, and J. Macfarlane, "Graph-Partitioning-Based Diffusion Convolutional Recurrent Neural Network for Large-Scale Traffic Forecasting," *Transportation Research Record*, vol. 2674, no. 9, 2020.
- [13] D. Andreoletti, S. Troia, F. Musumeci, S. Giordano, and G. Maier, "Network Traffic Prediction based on Diffusion Convolutional Recurrent Neural Networks," in *IEEE INFOCOM Workshops*, 2019.
- [14] P. Fafoutellis and E. I. Vlahogianni, "Unlocking the Full Potential of Deep Learning in Traffic Forecasting Through Road Network Representations: A Critical Review," *Data Science for Transportation*, vol. 5, p. 23, 2023.
- [15] Z. Qin et al., "A Survey on Modern Deep Learning Techniques for Traffic Volume Prediction," *Data Science and Engineering*, vol. 11, pp. 473-512, 2026.

- [16] W. Wang et al., "Edge-Cloud Collaborative Processing for Urban Traffic Streams," *Journal of Intelligent Transportation Systems*, vol. 27, pp. 112-128, 2023.
- [17] Y. Chen et al., "NoSQL Optimization for Large-Scale Trajectory Data Storage," *Data Science Journal*, vol. 18, 2024.
- [18] L. Zhao and H. Liu, "A Kafka-Spark Pipeline for Urban Mobility Monitoring," *IEEE Access*, vol. 10, pp. 45012-45025, 2022.
- [19] R. Singh et al., "Threshold-based Anomaly Detection in ITS using Stream Processing," *Transportation Research Part C*, vol. 140, 2022.
- [20] A. Gupta and S. Sharma, "MongoDB Sharding Strategies for High-Velocity IoT Telemetry Streams," *International Conference on Big Data*, 2021.